# Studio Nx

Backend(FastAPI/Python) 21 — signup() · unavailable_error()

2026-10-12

# Backend(FastAPI/Python) 21 — `signup()` · `unavailable_error()`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/auth_service.py` |
| 작업 | P1-04 회원가입: 가입 처리 (PRD 01 FR-101, FR-102, FR-108, 처리방침 동의) |
| 작성 | Claude (AI), 2026-10-07, 사용자 검토 |
| 이전 · 다음 | 20 `check_nickname()` · 22 `POST /auth/signup` |

## 1. 하는 일

join.html의 **Sign Up** 버튼으로 들어온 가입 요청을 검사하고, 통과하면 `users`에 새 회원을 저장합니다. 14번부터 20번까지를 모두 엮는 함수입니다.

| 함수 | 역할 |
|---|---|
| `signup()` | 검사하고 저장한 뒤, 저장된 `User`를 돌려줍니다 |
| `unavailable_error()` | 19번과 20번의 "쓸 수 없음" 결과를 409 오류로 바꿉니다 |

| 경우 | 결과 |
|---|---|
| 모두 통과 | 새 `User`를 돌려줍니다 (회원 번호와 가입 시각은 DB가 채웁니다) |
| 비밀번호 규칙 위반 | 422 `VALIDATION_ERROR` "Check the password requirements."와 `details.password: ["upper", …]` |
| 처리방침에 동의하지 않음 | 422 "Please agree to the Privacy Policy." |
| 예전 처리방침 화면에서 보냄 | 422 "Please reload the page." |
| 이메일이 사용 중 | 409 `EMAIL_TAKEN` "This email is already in use." |
| 닉네임이 사용 중 | 409 `NICKNAME_TAKEN` "This nickname is already in use." |
| 탈퇴한 지 3일이 안 됨 | 409 `REJOIN_TOO_SOON` "You can sign up with this email again after Oct 9, 2026 (UTC)." |

## 2. 언제 쓰나

`POST /api/v1/auth/signup`(22)이 이 함수를 부릅니다. 화면 흐름은 다음과 같습니다.

| 순서 | 화면 또는 서버 | 일어나는 일 |
|---|---|---|
| ① | join.html | 이메일과 닉네임의 **Check** 버튼(19, 20)을 눌러 ✓ Available을 확인합니다 |
| ② | join.html | 비밀번호 조건 목록이 모두 ✓가 되고, "만 16세 이상 + 처리방침 동의"에 체크합니다 |
| ③ | join.js (P1-07) | **Sign Up**을 누르면 `POST /auth/signup` `{email, nickname, password, privacy_agreed, privacy_version}`을 보냅니다 |
| ④ | **이 함수** | 같은 검사를 서버에서 한 번 더 하고 저장합니다 |
| ⑤ | join.js | 201을 받으면 홈으로 이동해서 로그인 모달을 엽니다 (P1-07) |

①과 ②를 화면에서 했더라도 ④에서 **다시 검사합니다**. 화면 검사는 건너뛸 수 있고, ①과 ③ 사이에 다른 사람이 그 이메일로 먼저 가입할 수도 있기 때문입니다.

## 3. 코드

**import** (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **`IntegrityError`, `AppError`, `ErrorCode`, `check_password_rules`, `hash_password`, `PRIVACY_VERSION`을 이번에 추가했습니다.**

| import | 가져오는 것 | 쓰는 곳 |
|---|---|---|
| `IntegrityError` | DB가 `UNIQUE` 같은 규칙을 어긴 요청을 거절할 때의 오류 | 동시 가입 처리 |
| `AppError`, `ErrorCode` | 01번의 오류 | 422, 409 |
| `check_password_rules`, `hash_password` | 14번, 15번 | 비밀번호 검사와 해시 |
| `PRIVACY_VERSION` | 지금 처리방침 버전 `"2026-10-05"` (`schemas/auth.py`) | 동의한 판 확인 |
| `SignupRequest` | 가입 요청의 모양 | `data` |

**함수**

```
def signup(db: Session, data: SignupRequest) -> User:
    failed = check_password_rules(data.password)
    if failed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Check the password requirements.",
                       details={"password": failed})
    if not data.privacy_agreed:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Please agree to the Privacy Policy.",
                       details={"privacy_agreed": "required"})
    if data.privacy_version != PRIVACY_VERSION:
        raise AppError(ErrorCode.VALIDATION_ERROR, 422, "Please reload the page.",
                       details={"privacy_version": "outdated"})

    email = normalize_email(data.email)
    nickname = data.nickname.strip()
    email_result = check_email(db, email, release=True)
    if not email_result.available:
        raise unavailable_error("email", email_result)
    nickname_result = check_nickname(db, nickname, release=True)
    if not nickname_result.available:
        raise unavailable_error("nickname", nickname_result)
```

```
    user = User(
        email=email,
        nickname=nickname,
        password_hash=hash_password(data.password),
        privacy_agreed_at=datetime.now(UTC).replace(tzinfo=None),
        privacy_version=PRIVACY_VERSION,
    )
    db.add(user)
    try:
        db.commit()
    except IntegrityError as exc:
        # 동시에 같은 값으로 가입한 사람이 먼저 저장된 경우 (위 검사는 둘 다 통과)
        db.rollback()
        if "uq_users_nickname" in str(exc.orig):
            field = "nickname"
        else:
            field = "email"
        raise unavailable_error(field, AvailabilityResponse(available=False, reason="TAKEN")) from None
    db.refresh(user)
    return user


def unavailable_error(field: str, result: AvailabilityResponse) -> AppError:
    if result.reason == "REJOIN_TOO_SOON":
        when = result.available_at
        day = f"{when:%b} {when.day}, {when.year}"  # 예: Oct 9, 2026
        return AppError(ErrorCode.REJOIN_TOO_SOON, 409,
                        f"You can sign up with this {field} again after {day} (UTC).",
                        details={field: "REJOIN_TOO_SOON", "available_at": when.isoformat()})
    if field == "email":
        return AppError(ErrorCode.EMAIL_TAKEN, 409, "This email is already in use.")
    return AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")
```

(실제 파일에서는 `ruff`가 `AppError(...)`의 인자를 한 줄에 하나씩 나눠 적었습니다. 내용은 같습니다.)

## 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `from sqlalchemy.exc import IntegrityError` | 라이브러리 안의 `exc`(오류 모음) 모듈에서 오류 종류 하나를 가져옵니다 |

| 식 | 뜻 |
|---|---|
| `if failed:` | 목록이 비어 있으면 거짓이고, 하나라도 있으면 참입니다 |
| `raise unavailable_error(…)` | 함수가 **만들어서 돌려준** 오류를 던집니다 (`unavailable_error`는 오류를 던지지 않고 돌려주기만 합니다) |
| `db.add(user)` → `db.commit()` | 저장을 대기시킨 뒤 확정합니다. 확정할 때 DB가 `UNIQUE`를 마지막으로 검사합니다 |
| `except IntegrityError as exc:` | DB가 거절하면 그 오류를 `exc`로 받습니다 |
| `str(exc.orig)` | MySQL이 보낸 원래 오류 문구입니다. `Duplicate entry 'newbie' for key 'users.uq_users_nickname'`처럼 어느 제약인지가 들어 있습니다 |
| `db.rollback()` | 실패한 저장을 취소합니다. 하지 않으면 세션을 더 쓸 수 없습니다 |

| 식 | 뜻 |
|---|---|
| `raise … from None` | DB 오류를 이어 붙이지 않고 이 프로젝트의 오류만 던집니다 (05번과 같음) |
| `db.refresh(user)` | DB가 채운 값(`user_id`, `created_at`, `role`, `status` …)을 다시 읽습니다 |
| `f"{when:%b}"` | 날짜를 영어 월 약자(`Oct`)로 씁니다 |
| `when.isoformat()` | `"2026-10-09T03:00:00+00:00"` 처럼, 화면(JS)이 읽기 쉬운 표준 모양으로 씁니다 |

## 4. 흐름

1. 비밀번호 규칙(14), 처리방침 동의, 처리방침 판을 확인합니다. **DB를 보지 않는** 검사를 먼저 합니다.
2. 이메일(19)과 닉네임(20)을 확인합니다. 3일이 지난 탈퇴 행이면 비켜 둡니다(18).
3. 새 `User`를 만듭니다. 이메일은 17번으로 정리하고, 비밀번호는 15번으로 해시하며, 동의 시각은 지금(UTC)으로 적습니다.
4. 저장(`commit`)합니다. DB가 거절하면(동시 가입) 409로 응답합니다.
5. DB가 채운 값을 읽어서 돌려줍니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| DB를 보지 않는 검사를 먼저 함 | 틀린 요청은 DB에 묻기 전에 끝냅니다. 비켜 두기(18)도 다른 검사를 모두 통과한 뒤에만 일어납니다 |
| 중간에 실패하면 `commit` 하지 않음 | 이메일을 비켜 둔 뒤 닉네임에서 막혀도, 요청이 끝날 때 세션이 닫히면서 비켜 둔 것까지 취소됩니다 |
| `role`, `status`, 용량은 넣지 않음 | DB 기본값(`USER`, `ACTIVE`, 5GB)이 정합니다 (08번과 같은 원칙) |
| `privacy_agreed_at`은 서버 시각으로 기록 | 화면이 보낸 시각은 믿을 수 없습니다. 동의 기록은 서버가 남깁니다 (처리방침 D3, D4) |
| `privacy_version` 확인 | 처리방침이 바뀐 뒤 예전 화면에서 보낸 가입이라면, 새 판을 보고 다시 동의하게 합니다 |
| `IntegrityError`를 409로 바꿈 | ①의 확인과 ③의 저장 사이에 다른 사람이 먼저 저장하면, 확인은 둘 다 통과합니다. 마지막에 DB의 `UNIQUE`가 막고, 그것을 500이 아니라 "이미 사용 중"으로 바꿔 응답합니다 |
| 날짜에 "(UTC)" 표시 | 서버는 사용자의 시간대를 모릅니다. 화면은 `details.available_at`으로 사용자 시간대의 날짜를 따로 보여 줄 수 있습니다 |
| 응답에 비밀번호나 해시를 넣지 않음 | 이 함수는 `User`를 돌려주지만, 라우터(22)는 `SignupResponse`(번호, 이메일, 닉네임)만 내보냅니다 |

**참고:** 닉네임의 2~20자 검사는 앞뒤 공백을 지우기 **전**의 길이로 합니다(스키마). 그래서 `" a "`는 3자로 통과한 뒤 `"a"`(1자)가 됩니다. 25번(입력 스키마)에서 공백을 먼저 지우고 길이를 재도록 고칩니다.

## 6. 시험 (`tests/test_auth_service.py`에 10개를 추가해서 모두 37개, 실제 MySQL로 실행)

| 시험 | 확인한 것 |
|---|---|
| 가입 성공 | 이메일은 소문자, 닉네임은 공백이 지워지고, 해시가 저장되며(원래 비밀번호가 아니고, 16번으로 맞음), `USER`와 `ACTIVE`, 동의 판과 시각이 기록되고, DB에 저장되어 있습니다 |

| 시험 | 확인한 것 |
|---|---|
| 비밀번호 규칙 | 422이고, `details.password = ["upper", "number", "special"]` 입니다 |
| 동의하지 않음, 예전 판 | 422와 각각의 문구 |
| 이메일 사용 중 | 409 `EMAIL_TAKEN` |
| 닉네임이 대소문자만 다름 | 409 `NICKNAME_TAKEN` |
| 탈퇴한 지 1일 | 409 `REJOIN_TOO_SOON`이고, 문구에 "Oct 9, 2026 (UTC)" 모양의 날짜와 `details.available_at`이 있습니다 |
| 탈퇴한 지 4일 | 같은 이메일과 닉네임으로 가입에 성공하고, 예전 행은 `deleted+…`와 `deleted_user_…`가 됩니다 |
| 동시 가입 (이메일, 닉네임 2개) | 확인을 통과한 것처럼 꾸민 뒤 저장하면 DB가 막고, 500이 아니라 409로 응답합니다 |

**시험 세션 변경 (`tests/conftest.py`):** 이 함수는 `commit`을 합니다. 그래서 시험의 `db` 세션을 "바깥 트랜잭션 + 저장점(SAVEPOINT)" 방식으로 바꿨습니다. 코드 안의 `commit`은 저장점만 확정하고, 시험이 끝나면 바깥 트랜잭션을 통째로 되돌립니다. 앞서 만든 모델과 서비스 시험도 모두 이 방식으로 통과합니다.

전체 시험 145개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.